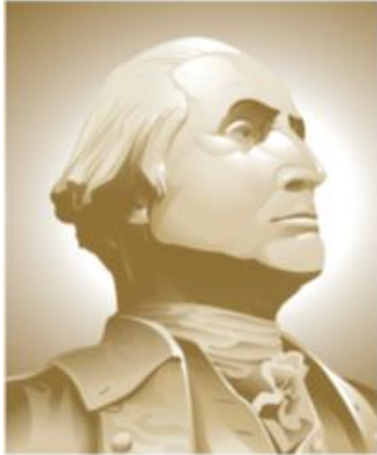




**THE GEORGE
WASHINGTON
UNIVERSITY**

WASHINGTON, DC

High-Performance Semantic Segmentation for Autonomous Urban Navigation

DATS_6303_10: Deep Learning

Individual Report

Shabnam Rafat Ummey Sawda

Instructor: Prof. Amir Jafari

1. Introduction:

Autonomous urban navigation represents the foundation of self-driving technology, enabling vehicles to interpret and interact with complex city environments without human intervention. The system must reliably process diverse urban scenes, identifying roads, vehicles, pedestrians, and infrastructure to make safe decisions in real-time. Semantic segmentation serves as the vehicle's visual comprehension system, analyzing images at the pixel level to classify every element in the scene. Unlike object detection which identifies bounding boxes, 2D semantic segmentation provides precise boundaries between elements, offering the granular interpretation essential for navigation decisions.

By implementing high-performance segmentation techniques, we wanted to explore how pixel-perfect scene understanding impacts vehicle decision-making processes. Through evaluating model performance across critical urban classes and challenging conditions, we gain crucial insights into both current limitations and potential enhancements. This work helped us to learn the concept of semantic segmentation, different models that are used for this purpose and also the application of 2D segmentation in autonomous urban navigation.

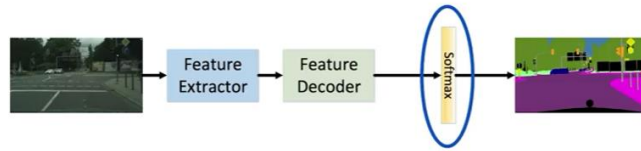
I find self-driving car technology really exciting. When learning how these cars work, I discovered they use methods like 2D and 3D semantic segmentation and object detection. Our goal wasn't just to finish a deep learning project; we wanted to start with one piece and keep building on it later. Though there are computing challenges, we aim to improve existing models and add more parts if possible. I looked through datasets from the project guidelines and found that Audi shares data for researchers to develop new technologies on their website. After downloading it, I ran some basic tests to see if we could complete the project on time. We realized 3D semantic segmentation would take too long, so our group focused only on the 2D part. We found a good starting point from AWS's GitHub page and created a script to map images with their labels. Working with my team, I planned how to make the basic model better and fix its problems. Then I built different models using various pre-trained networks and tracked how well they learned and performed over time.

2. Description of I work:

To get started, I needed to understand the basics of semantic segmentation and learn more about the Audi Autonomous Driving Dataset (A2D2). I did this by reading several published research papers, checking out articles on Medium, and taking a detailed Coursera course on autonomous driving offered by the University of Toronto. This helped me build a strong foundation of knowledge before diving into the actual project work.

A few snapshots from my learning journey are shared below:

Learning Same Resolution Feature Maps



Evaluation Metrics

- **True Positive (TP):** The number of correctly classified pixels belonging to class X
- **False Positive (FP):** The number of pixels that **do not belong** to class X in ground truth but **are classified** that class by the algorithm
- **False Negative (FN):** The number of pixels that **do belong** to class X in ground truth, but **are not classified** as that class by the algorithm

$$IOU_{class} = \frac{TP}{TP + FP + FN}$$

3. Describe the portion of the work:

First, I developed a custom data loader to map the raw images with their mask images. I later enhanced this with additional evaluation capabilities.

```

root_dir = '../Data/camera_lidar_semantic/'
output_file = 'a2d2_image_mask_pairs.csv'

pairs = []
for seq in sorted(os.listdir(root_dir)):
    seq_path = os.path.join(root_dir, seq)
    if not os.path.isdir(seq_path):
        continue

    # Path to camera images
    camera_path = os.path.join(seq_path, 'camera', 'cam_front_center')
    if not os.path.isdir(camera_path):
        continue

    # Path to label images
    label_path = os.path.join(seq_path, 'label', 'cam_front_center')
    if not os.path.isdir(label_path):
        continue

    # Find matching pairs
    for img_file in sorted(os.listdir(camera_path)):
        if not img_file.lower().endswith('.png'):
            continue

        img_path = os.path.join(camera_path, img_file)

        # Construct the expected label filename
        # Format: 20181204170238_camera_frontcenter_000130622.png -> 20181204170238_label_frontcenter_000130622.png
        label_file = img_file.replace(_old: 'camera', _new: 'label')

```

I began improving the model by updating the backbone to ResNet50, which is widely used because its deeper network with more layers can capture more complex features. Then I experimented with changing the pre-trained weights to COCO dataset since it's computationally less expensive and speeds up training. I tried various architectures including UNET and a hybrid approach using ImageNet+COCO pre-training. However, when tracking the performance metrics, these changes weren't showing meaningful improvements over time.

Next, I implemented DeepLabv3 Plus architecture, which is popular for semantic segmentation because its decoder module can extract finer details from images. Surprisingly, the results were actually worse than the simpler DeepLabv3 model. Unfortunately, we couldn't explore more fine-tuning options due to time constraints.

```

def create_deepLabv3plus_model(num_classes, encoder_name="resnet50", encoder_weights="imagenet", activation=None): 1 us3
    """
    Creates a DeepLabV3+ model using the segmentation-models-pytorch library.

    Args:
        num_classes: Number of output classes
        encoder_name: Name of the encoder backbone (default: "resnet50")
        encoder_weights: Pre-trained weights to use (default: "imagenet")
        activation: Activation function to use (default: None)

    Returns:
        A DeepLabV3+ model
    """
    try:
        # Try to load the model with specified parameters
        model = smp.DeepLabV3Plus(
            encoder_name=encoder_name,
            encoder_weights=encoder_weights,
            classes=num_classes,
            activation=activation
        )
        print(f"Successfully loaded DeepLabV3+ with {encoder_name} backbone using {encoder_weights} weights")
        return model
    except Exception as e:
        print(f"Error loading model: {e}")
        print("Falling back to default model")
        # Fallback to standard model with ResNet-50 backbone
        return smp.DeepLabV3Plus(
            encoder_name="resnet50",
            encoder_weights="imagenet",
            classes=num_classes,
            activation=activation
        )

```

I also implemented standard performance metrics to properly evaluate our models. I set up systems to save models after each training epoch and ensure all results and performance logs were recorded accurately. Finally, I developed a utilities script with helper functions and integrated it into all the training scripts I worked with.

4. Results:

The result with DeepLabv3 is shared below:

```
{
  "loss": 0.06953168307256123,
  "mean_iou": 0.38949742913246155,
  "pixel_acc": 0.9504124522209167,
  "mean_recall": 0.46915170550346375,
  "mean_precision": 0.5304993987083435,
  "mean_f1": 0.47310715913772583,
  "mean_dice": 0.47310715913772583,
  "bf_score": 0.47310715913772583
}
```

The model achieved excellent pixel accuracy (95%) but struggled with class-specific metrics, showing the classic semantic segmentation challenge of high overall accuracy with lower class balance. The mean IoU of 38.9% indicates room for improvement in consistently segmenting all classes. While recall is relatively strong at 46%, precision (53%) and combined metrics (F1, Dice, BF score all at 47.3%) suggest the model performs moderately well but could benefit from further optimization to better handle class imbalance.

The summary of classes identified is shared below:

- High-performing classes (IoU > 0.8):
0, 11, 43, 50, 51, 52
(mostly common classes)
- Poorly-performing classes (IoU < 0.2):
2, 3, 5-7, 9-10, 12-13, 15-16, 18
(mainly rare objects)
- Zero IoU classes:
12 classes have IoU of 0
(complete failure to detect)

The final model performance is shared below:

```
{
  "loss": 0.4547143042617038,
  "mean_iou": 0.19491489231586456,
  "pixel_acc": 0.6218803524971008,
  "mean_recall": 0.39099863171577454,
  "mean_precision": 0.30772536993026733,
  "f1_score":
  "mean_f1": 0.2770271301269531,
  "mean_dice": 0.2770271301269531,
  "bf_score": 0.2770271301269531,
```

The model seems significantly worse performance across all measures compared to the previous one. With a much higher loss (0.45) and substantially lower mean IoU (19.5%), this model struggles with class discrimination despite maintaining moderate pixel accuracy (62%). The precision and recall metrics are both weak (30.8% and 39.1% respectively), resulting in poor overall performance as reflected in the identical F1, Dice, and BF scores all at a disappointing 27.7%.

The summary of the classes identified is shared below:

- High-performing classes (IoU > 0.8):
0, 11, 17, 28, 33, 43, 51
(mostly common classes)
- Poorly-performing classes (IoU < 0.2):
3, 6, 7, 10, 13, 15, 16, 18, 24, 31, 37, 46, 53
(mainly rare objects)
- Zero IoU classes:
12 classes have IoU of 0
(complete failure to detect)

So, I tried other combinations as I mentioned. Few snapshots of the progress:

```
Epoch 0 - Batch 33/2751
Train Loss: 1.1971, Val Loss: 0.8876
Train mIoU: 0.0425, Val mIoU: 0.0483
Train F1: 0.0536, Val F1: 0.0587
Train Dice: 0.0536, Val Dice: 0.0587
Boundary F1: 0.0587, Samples/sec: 0.51
New best model saved with mIoU: 0.0483
Epoch 0 - Batch 34/2751
Train Loss: 1.1825, Val Loss: 0.8787
Train mIoU: 0.0427, Val mIoU: 0.0480
Train F1: 0.0538, Val F1: 0.0583
Train Dice: 0.0538, Val Dice: 0.0583
Boundary F1: 0.0583, Samples/sec: 0.51
Epoch 0 - Batch 35/2751
Train Loss: 1.1731, Val Loss: 0.8559
Train mIoU: 0.0437, Val mIoU: 0.0485
Train F1: 0.0553, Val F1: 0.0588
Train Dice: 0.0553, Val Dice: 0.0588
Boundary F1: 0.0588, Samples/sec: 0.51
New best model saved with mIoU: 0.0485
Epoch 0 - Batch 36/2751
Train Loss: 1.1596, Val Loss: 0.8495
Train mIoU: 0.0442, Val mIoU: 0.0483
Train F1: 0.0559, Val F1: 0.0586
Train Dice: 0.0559, Val Dice: 0.0586
Boundary F1: 0.0586, Samples/sec: 0.51
Epoch 0 - Batch 37/2751
Train Loss: 1.1412, Val Loss: 0.8444
Train mIoU: 0.0446, Val mIoU: 0.0478
Train F1: 0.0564, Val F1: 0.0580
Train Dice: 0.0564, Val Dice: 0.0580
Boundary F1: 0.0580, Samples/sec: 0.51
Epoch 0 - Batch 38/2751
Train Loss: 1.1303, Val Loss: 0.8336
Train mIoU: 0.0452, Val mIoU: 0.0477
Train F1: 0.0571, Val F1: 0.0580
Train Dice: 0.0571, Val Dice: 0.0580
Boundary F1: 0.0580, Samples/sec: 0.51
Epoch 0 - Batch 39/2751
Train Loss: 1.1286, Val Loss: 0.8167
Train mIoU: 0.0455, Val mIoU: 0.0484
Train F1: 0.0576, Val F1: 0.0587
Train Dice: 0.0576, Val Dice: 0.0587
Boundary F1: 0.0587, Samples/sec: 0.51
Epoch 0 - Batch 40/2751
Train Loss: 1.1224, Val Loss: 0.8061
Train mIoU: 0.0458, Val mIoU: 0.0486
Train F1: 0.0579, Val F1: 0.0590
Train Dice: 0.0579, Val Dice: 0.0590
Boundary F1: 0.0590, Samples/sec: 0.51
New best model saved with mIoU: 0.0486
```

```
Epoch 0 - Batch 107/2751
Train Loss: 0.7203, Val Loss: 0.4058
Train mIoU: 0.0644, Val mIoU: 0.0870
Train F1: 0.0808, Val F1: 0.1076
Train Dice: 0.0808, Val Dice: 0.1076
Boundary F1: 0.1076, Samples/sec: 0.51
Epoch 0 - Batch 108/2751
Train Loss: 0.7166, Val Loss: 0.4064
Train mIoU: 0.0646, Val mIoU: 0.0868
Train F1: 0.0811, Val F1: 0.1075
Train Dice: 0.0811, Val Dice: 0.1075
Boundary F1: 0.1075, Samples/sec: 0.51
Epoch 0 - Batch 109/2751
Train Loss: 0.7128, Val Loss: 0.4047
Train mIoU: 0.0650, Val mIoU: 0.0868
Train F1: 0.0815, Val F1: 0.1075
Train Dice: 0.0815, Val Dice: 0.1075
Boundary F1: 0.1075, Samples/sec: 0.51
```

```

Epoch 0 - Batch 376/3144
Train Loss: 0.4118, Val Loss: 0.1923
Train mIoU: 0.1096, Val mIoU: 0.1607
Train F1: 0.1428, Val F1: 0.2007
Train Dice: 0.1516, Val Dice: 0.2002
Boundary F1: 0.2002, Samples/sec: 0.51
Epoch 0 - Batch 377/3144
Train Loss: 0.4098, Val Loss: 0.1920
Train mIoU: 0.1098, Val mIoU: 0.1605
Train F1: 0.1431, Val F1: 0.2000
Train Dice: 0.1431, Val Dice: 0.2000
Boundary F1: 0.2000, Samples/sec: 0.51
Epoch 0 - Batch 378/3144
Train Loss: 0.4109, Val Loss: 0.1925
Train mIoU: 0.1101, Val mIoU: 0.1604
Train F1: 0.1431, Val F1: 0.2000
Train Dice: 0.1431, Val Dice: 0.2000
Boundary F1: 0.2000, Samples/sec: 0.51
Epoch 0 - Batch 379/3144
Train Loss: 0.4103, Val Loss: 0.1933
Train mIoU: 0.1101, Val mIoU: 0.1606
Train F1: 0.1431, Val F1: 0.2000
Train Dice: 0.1434, Val Dice: 0.2006
Boundary F1: 0.2006, Samples/sec: 0.51
Epoch 0 - Batch 380/3144
Train Loss: 0.4098, Val Loss: 0.1946
Train mIoU: 0.1101, Val mIoU: 0.1616
Train F1: 0.1435, Val F1: 0.2014
Train Dice: 0.1435, Val Dice: 0.2014
Boundary F1: 0.2014, Samples/sec: 0.51
Epoch 0 - Batch 381/3144
Train Loss: 0.4090, Val Loss: 0.1945
Train mIoU: 0.1101, Val mIoU: 0.1614
Train F1: 0.1437, Val F1: 0.2020
Train Dice: 0.1437, Val Dice: 0.2020
Boundary F1: 0.2020, Samples/sec: 0.51
Epoch 0 - Batch 382/3144
Train Loss: 0.4085, Val Loss: 0.1941
Train mIoU: 0.1104, Val mIoU: 0.1623
Train F1: 0.1439, Val F1: 0.2023
Train Dice: 0.1439, Val Dice: 0.2033
Boundary F1: 0.2033, Samples/sec: 0.51
Epoch 0 - Batch 383/3144
Train Loss: 0.4088, Val Loss: 0.1933
Train mIoU: 0.1104, Val mIoU: 0.1635
Train F1: 0.1440, Val F1: 0.2047
Train Dice: 0.1440, Val Dice: 0.2047
Boundary F1: 0.2047, Samples/sec: 0.51
Epoch 0 - Batch 384/3144
Train Loss: 0.4076, Val Loss: 0.1923
Train mIoU: 0.1109, Val mIoU: 0.1641
Train F1: 0.1442, Val F1: 0.2053
Train Dice: 0.1442, Val Dice: 0.2053
Boundary F1: 0.2053, Samples/sec: 0.51
New best model, saved with mIoU: 0.1641

```

```
Segmentation results saved to /home/ubuntu/ShabnamSawda/FinalProject/outputs/test_segmentation
All segmentation results saved to /home/ubuntu/ShabnamSawda/FinalProject/outputs
Generating class activation maps...
Generated 10 class activation maps in /home/ubuntu/ShabnamSawda/FinalProject/outputs/activation_maps
Analyzing class performance...

Potentially problematic classes:
- Class 5: IoU = 0.0000
  Confused with:
  - Class 4: 80.4%
  - Class 52: 6.2%
  - Class 43: 2.3%
- Class 6: IoU = 0.0000
  Confused with:
  - Class 4: 81.1%
  - Class 52: 6.9%
  - Class 38: 3.3%
- Class 7: IoU = 0.0000
  Confused with:
  - Class 45: 42.6%
  - Class 8: 35.2%
  - Class 4: 16.7%
- Class 10: IoU = 0.0000
  Confused with:
  - Class 8: 74.3%
  - Class 52: 9.6%
  - Class 45: 4.8%
- Class 12: IoU = 0.0000
  Confused with:
  - Class 11: 54.8%
  - Class 12: 12.0%
  - Class 43: 10.2%

Detailed performance analysis saved to /home/ubuntu/ShabnamSawda/FinalProject/outputs/class_performance.json
Performance report generated at /home/ubuntu/ShabnamSawda/FinalProject/outputs/performance_report.md
ubuntu@tp-10-1-3-6:~/ShabnamSawda/FinalProject/train$ export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

```
## Model Information

Architecture: deeplabv3plus_resnet50
Backbone: ResNet-50
Input Resolution: 480x480
Number of Classes: 55
Training Epochs: 3
Loss Function: focal

## Overall Performance Metrics

Mean IoU: 0.9347
Pixel Accuracy: 0.9504
Mean F1 Score: 0.4066
Mean Dice Coefficient: 0.4066
Boundary F1 Score: 0.4064

## Class-wise Performance

Class | IoU | Precision | Recall | F1 Score | Dice |
-----|-----|-----|-----|-----|-----|
Class 51 | 0.9801 | 0.9907 | 0.9892 | 0.9900 | 0.9900 |
Class 50 | 0.9558 | 0.9794 | 0.9794 | 0.9794 | 0.9794 |
Class 43 | 0.9294 | 0.9694 | 0.9575 | 0.9634 | 0.9634 |
Class 11 | 0.8765 | 0.9220 | 0.9467 | 0.9342 | 0.9342 |
Class 52 | 0.8617 | 0.9202 | 0.9313 | 0.9257 | 0.9257 |
Class 0 | 0.8060 | 0.8468 | 0.9436 | 0.8926 | 0.8926 |
Class 33 | 0.7928 | 0.8881 | 0.9014 | 0.8844 | 0.8844 |
Class 45 | 0.7489 | 0.8053 | 0.8591 | 0.8342 | 0.8342 |
Class 30 | 0.7451 | 0.7993 | 0.7180 | 0.8540 | 0.8540 |
Class 36 | 0.7240 | 0.8436 | 0.8362 | 0.8399 | 0.8399 |
Class 17 | 0.7187 | 0.7851 | 0.8948 | 0.8363 | 0.8363 |
Class 32 | 0.7088 | 0.7977 | 0.8618 | 0.8296 | 0.8296 |
Class 29 | 0.7076 | 0.8281 | 0.8375 | 0.8288 | 0.8288 |
Class 48 | 0.7024 | 0.8059 | 0.8872 | 0.8286 | 0.8286 |
Class 28 | 0.6939 | 0.8044 | 0.8324 | 0.8193 | 0.8193 |
Class 38 | 0.6344 | 0.7092 | 0.8574 | 0.7765 | 0.7765 |
```

5. Summary and Conclusions:

In this entire process, I've learned that performance isn't solely determined by having the most advanced architecture—it depends on proper tuning, configuration, and available computational resources. While I gained valuable knowledge about autonomous driving technologies, model architectures, and performance metrics, the practical challenges taught me the most important lessons. For example, I had to reduce image resolution by 50% to avoid persistent memory errors, and each training epoch still took an entire day. This experience highlighted real-world implementation challenges: in production environments where high-performance models must process full-resolution images across numerous epochs, balancing factors like learning rate schedulers and advanced backbones (such as ResNet-101) becomes critical.

Designing a network that properly balances computational requirements, performance needs, time constraints, and costs presents a genuinely complex challenge that goes well beyond theoretical understanding.

7. References:

<https://www.a2d2.audi/content/dam/a2d2/dataset/a2d2-audi-autonomous-driving-dataset.pdf>

<https://www.coursera.org/learn/visual-perception-self-driving-cars>

<https://medium.com/@itberrios6/deeplabv3-c0c8c93d25a4#e9e8>

<https://sh-tsang.medium.com/review-deeplabv3-atrous-separable-convolution-semantic-segmentation-a625f6e83b90>

<https://medium.com/@rlj1nghimire/semantic-segmentation-using-deeplabv3-from-scratch-b1ff57a27be>